



HSB

Hochschule Bremen
City University of Applied Sciences

HOCHSCHULE BREMEN
FAKULTÄT 4 - ELEKTROTECHNIK UND INFORMATIK
INTERNATIONALER STUDIENGANG MEDIENINFORMATIK (B.Sc.)

EXPOSÉ

Nachhaltige Persistenz in Webanwendungen

– **Eine Analyse und Implementation verschiedener
Methoden von Datenspeicherung anhand eines
Planspiels** –

Marvin Uwe Marken (*373116*)

21. Mai 2025 (Version 1.00)

1 Einleitung

2023 wurde ein Planspiel zum Thema “Diversität in Unternehmen” entwickelt. Dieses gewann im November 2023 zwei Preise bei der *Diversity Challenge*, einem Wettbewerb, welcher von dem Charta der Vielfalt e. V. organisiert wird. Dort noch in Form eines physischen Kartenspiels, begann man im Anschluss zum Wettbewerb mit der Entwicklung einer digitalen Variante in Form einer Webanwendung. Die Anwendung ist mittlerweile auf einem vollwertig durchführbaren Stand. Bisher ist es aber nur allein aus der Sicht des Spielleiters spielbar und der Fortschritt der Sitzung geht nach Verbindungsverlust verloren. In dieser Arbeit soll der Fortschrittsverlust durch persistente Speicherung der Sitzungsdaten behoben werden. Dabei wird untersucht, wie die Persistenz möglichst nachhaltig gestaltet werden kann.

2 Problemstellung und Lösungsansatz

2.1 Problemstellung

In der Entwicklung moderner Webanwendungen ist die Suche nach einer geeigneten Methodik zur effizienten Speicherung von Nutzerdaten immer wieder eine Herausforderung. Die Wahl der Persistenz-Methode beeinflusst maßgeblich die Performance, Skalierbarkeit und Benutzererfahrung der Anwendung, aber auch dessen Nachhaltigkeit. Dies kommt durch die geplante Mehrspielerfunktion umso mehr zum Tragen, da Live-Updates der Daten durchgeführt werden müssen.

Ein Mehrspielerspiel erfordert eine Datenhaltung, die schnelle Lese- und Schreibzugriffe ermöglicht, um Echtzeitinteraktionen zwischen Spielern zu unterstützen. Gleichzeitig müssen Persistenz-Lösungen sicherstellen, dass Spielstände zuverlässig gespeichert und bei Unterbrechungen korrekt wiederhergestellt werden können. Herkömmliche Methoden, die nur darauf ausgelegt sind, die Interaktionen eines einzelnen Nutzers zu speichern, reichen hier nicht aus. Um eine gute Nachhaltigkeit zu erreichen, sollten zudem auch große Datenverarbeitungen vermieden werden, wie z.B. die gesamte Spielsitzung bei allen Spielern nach jeder Interaktion zu aktualisieren oder Daten(-strukturen) zu wiederholen, die vermieden werden können. Folgende Leitfragen (ab hier als *LF* abgekürzt) werden hierbei gestellt und im Laufe der Arbeit behandelt:

- LF1** Wie beeinflusst die Wahl der Persistenz-Technologie (relationale gegenüber nicht-relationalen Datenbanken) die Skalierbarkeit des Planspiels?
- LF2** Welche Technologie bietet die beste Performance bzgl. Lese-/Schreibgeschwindigkeit und Fehleranfälligkeit für die Persistenz von Echtzeitdaten für eine spätere Mehrspielerfunktion?
- LF3** Wie kann die Persistenz in dem Planspiel ressourcenschonend und energieeffizient gestaltet werden?

2.2 Lösungsansatz

Eine simple Lösung wäre zu bestimmten Zeitpunkten den aktuellen Spielstand wieder mit demselben Aufbau der zwei JSON-Dateien, zu speichern, sodass diese beim Verlassen und Wiedereinstieg in das Planspiel anstelle der originalen Dateien direkt geladen werden können. Dabei ist es wichtig zu entscheiden, ob diese auf dem Server oder lokal beim Spieler gespeichert wird. Auf dem Server wäre ohne eine Implementation einer Sitzungsverwaltung nur eine Sitzung zurzeit möglich, dafür wäre der aktuelle Stand von allen Spielern jederzeit abrufbar. Lokal bei dem Spieler zu speichern, hätte den Vorteil, dass keine Sitzungsverwaltung benötigt wird, da beim Aufrufen des Planspiels ein eigener Aufbau geladen wird. Der Nachteil wäre, dass andere Spieler keinen Zugriff auf den Spielstand des anderen haben und somit das Spiel nur in Form einer Einzelspielererfahrung bzw. als Präsentation des Spielers genutzt werden kann. Alternativ wäre auch eine hybride Variante möglich, sodass auf beiden Seiten der Spielstand gespeichert wird und man so nur noch die Unterschiede zum

Spielstand auf dem Server aktualisieren muss.

Um die Möglichkeit einer Mehrspielerfunktion zu bewahren, sollte eine Spielsitzung auf einem Server erfolgen. Dabei kann man Entweder den Webserver nutzen, auf dem auch das Planspiel läuft oder über Technologien wie Peer-to-Peer z. B. den Spielleiter als Server definieren.

Dies bietet aber keinerlei Datenschutz, da die Dateien in dieser Form, ohne benötigten Login, vollständig öffentlich abrufbar wären. Im Gegenzug dazu ist es bei einer Datenbank üblich, nur Teile davon über eine API¹, unter vorherigem Login, zur Verfügung zu stellen. Dennoch kann diese Methode zu Beginn als Vergleichsgrundlage dienen. Danach wird die Datenstruktur in relationale Datenbanken (basierend auf SQLite (?)), nicht-relationale Datenbanken (basierend auf MongoDB (?)) und (vielleicht?) einer hybriden Version erneut umgesetzt und verglichen.

Verglichen werden Ladezeiten, Datengrößen und Energieverbrauch. Zum Erheben der Metriken werden Google Chrome DevTools genutzt. Dort können unter anderem im Reiter "Netzwerk" die Ladezeiten und Datengrößen gemessen werden. (Tool zum Messen des Energieverbrauchs finden und definieren)

¹Application Programming Interface

3 Literaturübersicht

- **Angular 16 Dokumentation**[1] In dieser Dokumentation wird das vom Planspiel genutzte Framework Angular 16 erklärt.

4 Tools

Hardware:

- Notebook - Lenovo ThinkPad P15v Gen 1
- CPU - Intel Core i7-10875H
- RAM - 64GB DDR4
- Server - (Bei den HEC-Admins anfragen?)

Implementationstools:

- JetBrains WebStorm Entwicklungsumgebung
- Google Chrome DevTools

Software-Bibliotheken:

- Angular
- SQLite (?)
- MongoDB (?)
- Docker

Versionsverwaltung:

- Git - Versionskontrollsoftware

5 Vorläufige Gliederung

Nachfolgend die vorläufige Gliederung der Thesis. Es gilt zu beachten, dass vor allem die prototypische Realisierung sich radikal von der Planung unterscheiden kann.

Eigenständigkeitserklärung

Abstract

1. Einleitung

1.1. Problemfeld

1.2. Ziele der Arbeit

1.3. Lösungsansatz

1.4. Aufbau der Arbeit

2. Grundlagen

2.1. Angular

2.2. Datenbanken

2.2.1. Relationale Datenbanken

2.2.2. Nicht-Relationale Datenbanken

2.3. Docker

3. Verwandte Arbeiten

4. Konzeption

5. Prototypische Realisierung

5.1. Wahl der Realisierungsplattform

5.2. Festlegung des Realisierungsumfangs

5.3. Ausgewählte Realisierungsaspekte

5.4. Qualitätssicherung

5.5. Zusammenfassung

6. Evaluation

6.1. Überprüfung funktionaler Anforderungen

6.2. Überprüfung nicht-funktionaler Anforderungen

7. Zusammenfassung und Ausblick

7.1. Zusammenfassung

7.2. Ausblick

Literaturverzeichnis

6 Zeitplanung

Geplanter Starttermin: 31.03.2025

Bearbeitungsdauer: 9 Wochen

Tabelle 1 stellt die geplanten Arbeitspakete und Meilensteine dar:

Tabelle 1: Arbeitspakete und Meilensteine

M1	Offizieller Beginn der Arbeit	31.03.2025
	• Verfassen von Kapitel 1 (Einleitung)	1 Woche
M2	Recherche & Grundlagen	07.04.2025
	• Recherche • Verfassen von Kapitel 2 (Grundlagen) • Verfassen von Kapitel 3 (Verwandte Arbeiten)	2 Wochen
M3	Konzeption	21.04.2025
	• Konzeption • Verfassen von Kapitel 4 (Konzeption)	2 Wochen
M4	Implementation	05.05.2025
	• Implementation • Verfassen von Kapitel 5 (Realisierung)	2 Wochen
M5	Evaluation	19.05.2025
	• Verfassen von Kapitel 6 (Evaluation) • Verfassen von Kapitel 7 (Zusammenfassung & Ausblick)	1 Woche
M6	Erste Fassung vollständige Thesis	26.05.2025
	• Korrekturlesen • Drucken & binden lassen	1 Woche
M7	Abgabe der Thesis	02.06.2025

Literatur

Online-Quellen

- [1] Ohne angegebenen Verfasser. *Angular - Introduction to the Angular docs*. Google. URL: <https://v16.angular.io/docs> (besucht am 26.03.2025).